

Phase 8 Structured Note: Dividing Functions with Two Calls and Linear Work

1. Current Progress Analysis

Before Phase 8, you already learned how to turn recursive code into a recurrence relation and solve several common recurrence patterns.

You have already studied:

Phase	Main idea	Example recurrence	Final result
Phase 1	Turn recursive code into a recurrence relation	$T(n) = T(n - 1) + 1$	$O(n)$
Phase 2	Solve decreasing one-call recurrences	$T(n) = T(n - 1) + 1$	$O(n)$
Phase 3	Handle two decreasing recursive calls	$T(n) = 2T(n - 1) + 1$	$O(2^n)$
Phase 4	Use the decreasing master theorem	$T(n) = aT(n - b) + f(n)$	depends on a , b , and $f(n)$
Phase 5	Divide input by 2 with constant work	$T(n) = T(n / 2) + 1$	$O(\log n)$
Phase 6	Divide input by 2 with linear work	$T(n) = T(n / 2) + n$	$O(n)$
Phase 7	Two half-size calls with constant work	$T(n) = 2T(n / 2) + 1$	$O(n)$

Big idea from Phase 5

If the input is divided by 2 each time, the number of recursion levels is $\log n$.

Example:

8 → 4 → 2 → 1

So:

$$T(n) = T(n / 2) + 1 \rightarrow O(\log n)$$

Big idea from Phase 6

Even if there are only $\log n$ levels, we must still count how much work happens at each level.

For example:

$$T(n) = T(n / 2) + n$$

The work is:

$$n + n/2 + n/4 + n/8 + \dots$$

That sum is about $2n$, so:

$$T(n) = O(n)$$

Big idea from Phase 7

Two recursive calls do not always mean exponential time.

Phase 7 studied:

$$T(n) = 2T(n / 2) + 1$$

This gives:

$$O(n)$$

Why?

Because even though each call makes two more calls, the input is divided by 2, so the recursion tree has only $\log n$ levels.

What changes in Phase 8?

Phase 8 keeps the same two half-size recursive calls from Phase 7, but the current work changes.

Phase 7:

$$T(n) = 2T(n / 2) + 1$$

Phase 8:

$$T(n) = 2T(n / 2) + n$$

The only change is:

+1 changes to +n

That small change is important.

It changes the final answer from:

$$O(n)$$

to:

$$O(n \log n)$$

2. Main Goal of Phase 8

The goal of Phase 8 is to understand recursive functions that:

1. Do linear work in the current call.
2. Make two recursive calls.
3. Give each recursive call half of the input.

The main recurrence is:

$$T(n) = 2T(n / 2) + n$$

The final time complexity is:

$$O(n \log n)$$

The stack space is:

$O(\log n)$

Main lesson

Two half-size recursive calls plus linear work gives:

$O(n \log n)$

This is one of the most important recurrence patterns in data structures and algorithms.

It is the same recurrence used by **merge sort**.

3. What Is a Dividing Function with Two Calls and Linear Work?

A **dividing function** is a recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

This means the next recursive call receives half of the current input.

In Phase 8, the function does this twice:

```
algorithm(n / 2);  
algorithm(n / 2);
```

So one function call creates two smaller calls.

Each smaller call receives:

$n / 2$

That gives the recursive part:

$2T(n / 2)$

The function also does a loop of size **n**:

```
for (int i = 1; i <= n; i++) {  
    cout << i << " ";  
}
```

This loop runs `n` times, so the current work is:

`+n`

Putting both ideas together:

$$T(n) = 2T(n / 2) + n$$

4. Main Code Example

```
#include <iostream>  
using namespace std;  
  
void algorithm(int n) {  
    if (n > 1) {  
        // Linear work at this level: O(n)  
        for (int i = 1; i <= n; i++) {  
            cout << i << " ";  
        }  
  
        cout << "| ";  
  
        // Two recursive calls, each on half of n  
        algorithm(n / 2);  
        algorithm(n / 2);  
    }  
}  
  
int main() {  
    int n = 4;  
    cout << "Output: ";  
    algorithm(n);  
    cout << endl;  
    return 0;  
}
```

5. What the Code Does

The function is:

```
void algorithm(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }

        cout << "| ";

        algorithm(n / 2);
        algorithm(n / 2);
    }
}
```

It does four main things:

1. It checks whether `n > 1`.
2. If true, it runs a loop from `1` to `n`.
3. It calls itself once with `n / 2`.
4. It calls itself again with `n / 2`.

The function stops when `n` becomes `1` or smaller.

6. Breaking the Code Into Parts

Look at the important part:

```
if (n > 1) {
    for (int i = 1; i <= n; i++) {
        cout << i << " ";
    }

    algorithm(n / 2);
    algorithm(n / 2);
}
```

Inside each working call, there are three important parts.

Part 1: Base Condition

```
if (n > 1)
```

This means the function continues only while `n` is greater than `1`.

When `n = 1`, the condition becomes false:

```
1 > 1 is false
```

So the function stops.

The base case is:

```
T(1) = 1
```

This means the stopping case takes constant time.

Part 2: Current Work

```
for (int i = 1; i <= n; i++) {  
    cout << i << " ";  
}
```

This loop runs from `1` to `n`.

So it runs `n` times.

Therefore, the current work is:

```
n
```

This is called **linear work**.

So:

```
f(n) = n
```

Part 3: First Recursive Call

```
algorithm(n / 2);
```

This call gives half of the input to the next recursive call.

Its time is:

```
T(n / 2)
```

Part 4: Second Recursive Call

```
algorithm(n / 2);
```

This is another recursive call with the same input size.

Its time is also:

```
T(n / 2)
```

So the two recursive calls together give:

```
T(n / 2) + T(n / 2)
```

That simplifies to:

```
2T(n / 2)
```

7. Writing the Recurrence Relation

The total time is:

```
first recursive call time  
+ second recursive call time  
+ current work
```

The first recursive call is:

$$T(n / 2)$$

The second recursive call is:

$$T(n / 2)$$

The current work is:

$$n$$

So:

$$T(n) = T(n / 2) + T(n / 2) + n$$

Since the two recursive calls are the same size:

$$T(n) = 2T(n / 2) + n$$

Base case:

$$T(1) = 1$$

Full recurrence:

$$\begin{aligned} T(n) &= 2T(n / 2) + n, \text{ for } n > 1 \\ T(1) &= 1 \end{aligned}$$

8. Meaning of Each Part of the Recurrence

The recurrence is:

$$T(n) = 2T(n / 2) + n$$

Part	Meaning
$T(n)$	Time taken for input size n
2	There are two recursive calls
$T(n / 2)$	Each recursive call receives half the input
$+ n$	Linear work done in the current call
$T(1) = 1$	Base case takes constant time

In simple words:

The function does n work now, then creates two smaller problems, each of size $n / 2$.

9. General Division Recurrence Form

Dividing recurrences often follow this form:

$$T(n) = aT(n / b) + f(n)$$

Where:

Symbol	Meaning
a	Number of recursive calls
b	Division factor
$f(n)$	Extra work outside recursion

For Phase 8:

$$T(n) = 2T(n / 2) + n$$

So:

Symbol	Value	Reason
a	2	There are two recursive calls
b	2	The input is divided by 2
$f(n)$	n	The loop runs n times

So Phase 8 has:

```
a = 2  
b = 2  
f(n) = n
```

10. Dry Run for `algorithm(4)`

Call:

```
algorithm(4);
```

Step 1: First call, `algorithm(4)`

Since `4 > 1` is true, the loop runs from `1` to `4`.

It prints:

```
1 2 3 4
```

That is `4` units of work.

Then it calls:

```
algorithm(2)  
algorithm(2)
```

Step 2: First `algorithm(2)`

Since `2 > 1` is true, the loop runs from `1` to `2`.

It prints:

```
1 2
```

That is `2` units of work.

Then it calls:

```
algorithm(1)
algorithm(1)
```

Both `algorithm(1)` calls stop because:

```
1 > 1 is false
```

Step 3: Second `algorithm(2)`

The second `algorithm(2)` also prints:

```
1 2
```

That is another `2` units of work.

Then it calls:

```
algorithm(1)
algorithm(1)
```

Both stop.

Output Pattern for `n = 4`

The output pattern is:

```
1 2 3 4 | 1 2 | 1 2 |
```

Work Table for `n = 4`

Level	Number of calls	Size of each call	Work per call	Total work at level
Level 0	1	4	4	4
Level 1	2	2	2	4

Level	Number of calls	Size of each call	Work per call	Total work at level
Stop level	4	1	0 or constant stop work	ignored in Big O

Important observation:

Level 0 total work = 4
Level 1 total work = 4

Each working level costs the same total amount:

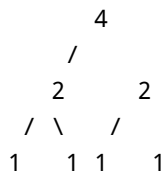
n

For $n = 4$, each working level costs 4.

11. Recursion Tree Intuition

The recursive calls form a tree.

For $n = 4$, the tree looks like this:



The 1 calls stop because they reach the base case.

Each working node does work equal to its input size.

So:

Level 0: one call of size 4
Total work = 4

Level 1: two calls of size 2
Total work = $2 + 2 = 4$

Each level costs 4, which is the original n .

12. Level-by-Level Intuition for General n

For general input n , the recursion tree works like this:

Level	Number of calls	Size of each call	Work per call	Total work at level
Level 0	1	n	n	n
Level 1	2	$n / 2$	$n / 2$	$2 \times (n / 2) = n$
Level 2	4	$n / 4$	$n / 4$	$4 \times (n / 4) = n$
Level 3	8	$n / 8$	$n / 8$	$8 \times (n / 8) = n$
Level i	2^i	$n / 2^i$	$n / 2^i$	$2^i \times (n / 2^i) = n$

Key idea

At every level:

$$\text{number of calls} \times \text{work per call} = n$$

So each level costs:

$$n$$

13. Why There Are $\log n$ Levels

The input keeps being divided by 2:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

This means after k levels, the input size is:

$$n / 2^k$$

The recursion stops when the input reaches 1:

$$n / 2^k = 1$$

Solve this equation:

$$\begin{aligned} n / 2^k &= 1 \\ n &= 2^k \\ k &= \log_2(n) \end{aligned}$$

So the number of levels is:

$$\log_2(n)$$

In Big O notation, we usually write:

$$\log n$$

14. Why the Final Answer Is $O(n \log n)$

Now combine the two important facts:

Each level costs n .
There are $\log n$ levels.

So total work is:

$$n + n + n + \dots + n$$

This happens $\log n$ times.

Therefore:

$$\text{Total work} = n \times \log n$$

Final time complexity:

$$T(n) = O(n \log n)$$

15. Why the Answer Is Not $O(n)$

A beginner might think:

Each level costs n , so the answer is $O(n)$.

But that is not enough.

There is not just one level.

There are:

$\log n$ levels

So the total is:

$n \times \log n$

Therefore, the answer is not:

$O(n)$

It is:

$O(n \log n)$

16. Why the Answer Is Not $O(n^2)$

A beginner might think:

There is a loop and two recursive calls, so it must be $O(n^2)$.

But this is not correct.

The two recursive calls are not both size n .

They are each size:

$$n / 2$$

Then those calls become:

$$n / 4$$

Then:

$$n / 8$$

The input keeps shrinking.

The work spreads across levels, and each level costs only n , not n^2 .

Since there are $\log n$ levels, the answer is:

$$O(n \log n)$$

which is smaller than $O(n^2)$.

17. Successive Substitution Method

Now we solve the recurrence formally.

We solve:

$$T(n) = 2T(n / 2) + n$$

Step 1: Start with the recurrence

$$T(n) = 2T(n / 2) + n$$

Step 2: Substitute $T(n / 2)$

Using the same recurrence rule:

$$T(n / 2) = 2T(n / 4) + n/2$$

Substitute it into the original recurrence:

$$T(n) = 2[2T(n / 4) + n/2] + n$$

Multiply:

$$T(n) = 4T(n / 4) + n + n$$

So:

$$T(n) = 2^2T(n / 4) + 2n$$

Step 3: Substitute again

Using the same rule:

$$T(n / 4) = 2T(n / 8) + n/4$$

Substitute:

$$T(n) = 2^2[2T(n / 8) + n/4] + 2n$$

Multiply:

$$T(n) = 2^3T(n / 8) + n + 2n$$

So:

$$T(n) = 2^3T(n / 8) + 3n$$

Step 4: See the pattern

After 1 step:

$$T(n) = 2T(n / 2) + n$$

After 2 steps:

$$T(n) = 2^2T(n / 4) + 2n$$

After 3 steps:

$$T(n) = 2^3T(n / 8) + 3n$$

After steps:

$$T(n) = 2^kT(n / 2^k) + kn$$

This is the general pattern.

18. Reaching the Base Case

The base case is:

$$T(1) = 1$$

So we want the input inside to become .

The recursive part is:

$$T(n / 2^k)$$

We want:

$$n / 2^k = 1$$

Now solve this equation:

$$\begin{aligned}n / 2^k &= 1 \\n &= 2^k \\k &= \log_2(n)\end{aligned}$$

So the recursion reaches the base case after:

$$\log_2(n)$$

levels.

19. Substitute Back

The pattern is:

$$T(n) = 2^k T(n / 2^k) + kn$$

At the base case:

$$n / 2^k = 1$$

So:

$$T(n / 2^k) = T(1)$$

Then:

$$T(n) = 2^k T(1) + kn$$

Since:

$$T(1) = 1$$

we get:

$$T(n) = 2^k + kn$$

But we know:

$$2^k = n$$

and:

$$k = \log_2(n)$$

So:

$$T(n) = n + n\log_2(n)$$

In Big O notation:

$$T(n) = O(n \log n)$$

20. Full Solution in One Place

Start:

$$T(n) = 2T(n/2) + n$$

Substitute once:

$$\begin{aligned} T(n) &= 2[2T(n/4) + n/2] + n \\ T(n) &= 2^2T(n/4) + 2n \end{aligned}$$

Substitute again:

$$T(n) = 2^3T(n/8) + 3n$$

Pattern:

$$T(n) = 2^kT(n/2^k) + kn$$

Use the base case:

$$n / 2^k = 1$$

Solve:

$$\begin{aligned} n &= 2^k \\ k &= \log_2(n) \end{aligned}$$

Substitute into the pattern:

$$T(n) = 2^k T(1) + kn$$

Since $T(1) = 1$:

$$T(n) = 2^k + kn$$

Since $2^k = n$ and $k = \log_2(n)$:

$$T(n) = n + n \log_2(n)$$

Final Big O:

$$T(n) = O(n \log n)$$

21. Beginner-Friendly Meaning of the Solution

The recurrence:

$$T(n) = 2T(n / 2) + n$$

means:

Do n work now.
Then solve two smaller problems.
Each smaller problem has size $n / 2$.

The recursion tree gets wider:

```
1 call
2 calls
4 calls
8 calls
...
```

But the input size gets smaller:

```
n
n/2
n/4
n/8
...
1
```

At every level, the total work stays n :

```
n
n
n
n
...
```

There are $\log n$ levels.

So the total is:

```
n + n + n + ... for log n levels
```

Final result:

```
 $O(n \log n)$ 
```

22. Merge Sort Connection

This recurrence is famous because it matches **merge sort**.

Merge sort does two main things:

1. It divides the array into two halves.
2. It merges the two sorted halves in linear time.

The recurrence is:

$$T(n) = 2T(n / 2) + n$$

Meaning:

$2T(n / 2)$ = sort the two halves
 $+n$ = merge the two halves

So merge sort has time complexity:

$$O(n \log n)$$

Simple merge sort meaning

If you have an array of size n :

Split into left half and right half.
Sort left half.
Sort right half.
Merge both halves.

Sorting the two halves gives:

$$2T(n / 2)$$

Merging takes:

n

So:

$$T(n) = 2T(n / 2) + n$$

23. Time Complexity

The time complexity is:

$O(n \log n)$

Why?

Because:

Each level costs n .
There are $\log n$ levels.

So:

$n \times \log n = n \log n$

24. Stack Space Complexity

The recursion stack space is:

$O(\log n)$

Why?

Even though the recursion creates many total calls, the computer does not keep all calls active at the same time.

The deepest active path looks like this:

```
algorithm(n)
algorithm(n / 2)
algorithm(n / 4)
algorithm(n / 8)
...
algorithm(1)
```

The length of this path is:

$$\log_2(n)$$

So the stack space is:

$$O(\log n)$$

Important difference

Total work = $O(n \log n)$
Stack depth = $O(\log n)$

The total work counts all nodes and work in the recursion tree.

The stack space counts only the deepest active path at one time.

25. Time Complexity vs Stack Space

For Phase 8:

Recurrence	Time complexity	Stack space
$T(n) = 2T(n/2) + n$	$O(n \log n)$	$O(\log n)$

Why time is $O(n \log n)$:

Each level costs n .
There are $\log n$ levels.

Why stack space is $O(\log n)$:

The deepest path keeps dividing n by 2 until reaching 1.

26. Comparison with Phase 5, Phase 6, and Phase 7

Feature	Phase 5	Phase 6	Phase 7	Phase 8
Recurrence	$T(n)=T(n/2)+1$	$T(n)=T(n/2)+n$	$T(n)=2T(n/2)+1$	$T(n)=2T(n/2)+n$
Recursive calls	1	1	2	2
Input change	$n/2$	$n/2$	$n/2$	$n/2$
Extra work	constant	linear	constant	linear
Main work idea	$1 + 1 + 1 + \dots$	$n + n/2 + n/4 + \dots$	$1 + 2 + 4 + \dots$	$n + n + n + \dots$
Number of levels	$\log n$	$\log n$	$\log n$	$\log n$
Time complexity	$O(\log n)$	$O(n)$	$O(n)$	$O(n \log n)$
Stack space	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Key lesson

All four phases divide the input by 2, so all have $\log n$ stack depth.

But their time complexities differ because:

1. The number of recursive calls may differ.
2. The work done at each call may differ.

27. Important Terms in Phase 8

Dividing Function

A recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

Division Recurrence

A recurrence where the recursive part contains division.

Example:

$$T(n) = 2T(n / 2) + n$$

Linear Work

Work that grows directly with `n`.

Example:

```
for (int i = 1; i <= n; i++) {  
    cout << i << " ";  
}
```

This loop runs `n` times, so it is linear work.

Two Recursive Calls

The function calls itself twice.

Example:

```
algorithm(n / 2);  
algorithm(n / 2);
```

This gives:

$$2T(n / 2)$$

Base Case

The condition where recursion stops.

In Phase 8:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Recursion Tree

A tree diagram that shows how recursive calls split.

For Phase 8:

```
      n
     /
    n/2
   /  \
  n/4  n/4
 /  \  /  \
n/4 n/4 n/4 n/4
```

The tree gets wider, but the input size gets smaller.

Level Cost

Level cost means the total work done at one level of the recursion tree.

For Phase 8, every level costs:

```
n
```

Logarithmic Depth

The recursion depth is the number of times we divide `n` by 2 until we reach 1.

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

So the depth is:

$O(\log n)$

$n \log n$ **Time**

$n \log n$ means:

n work per level $\times \log n$ levels

This is the result of Phase 8.

28. Complete Coding Program

```
#include <iostream>
using namespace std;

// This function does linear work, then calls itself twice with n / 2.
void algorithm(int n) {
    if (n > 1) {
        // Linear work at this level: O(n)
        for (int i = 1; i <= n; i++) {
            cout << i << " ";
        }

        cout << "| ";

        // First recursive call on half input
        algorithm(n / 2);

        // Second recursive call on half input
        algorithm(n / 2);
    }
}

int main() {
    int n = 4;
```

```
    cout << "Output: ";  
    algorithm(n);  
    cout << endl;  
  
    return 0;  
}
```

Output for `n = 4`

Output: 1 2 3 4 | 1 2 | 1 2 |

Recurrence

$$T(n) = 2T(n / 2) + n$$
$$T(1) = 1$$

Time complexity

$$O(n \log n)$$

Stack space

$$O(\log n)$$

29. Common Beginner Mistakes

Mistake 1: Saying the answer is `O(n)`

Wrong idea:

Each level costs n , so the answer is $O(n)$.

Correct idea:

Each level costs n , but there are $\log n$ levels.

So:

$$n \times \log n = O(n \log n)$$

Mistake 2: Saying the answer is $O(n^2)$

Wrong idea:

There is a loop and two recursive calls, so the answer must be $O(n^2)$.

Correct idea:

The recursive calls are on $n / 2$, not n .

The input keeps shrinking.

So the answer is:

$$O(n \log n)$$

not:

$$O(n^2)$$

Mistake 3: Confusing Phase 7 and Phase 8

Phase 7:

$$T(n) = 2T(n / 2) + 1 \rightarrow O(n)$$

Phase 8:

$$T(n) = 2T(n / 2) + n \rightarrow O(n \log n)$$

The only difference is:

+1 versus +n

That difference changes the answer.

Mistake 4: Thinking every dividing recurrence is $O(\log n)$

Wrong idea:

The input is divided by 2, so the answer must be $O(\log n)$.

Correct idea:

Dividing by 2 tells us the number of levels.

But we must also count the work at each level.

In Phase 8:

There are $\log n$ levels.
Each level costs n .

So:

$O(n \log n)$

Mistake 5: Forgetting the base case equation

For division recurrences, the base case equation is usually:

$n / 2^k = 1$

This gives:

$k = \log_2(n)$

Do not use the decreasing recurrence equation:

$$n - k = 0$$

That is for decreasing recurrences, not dividing recurrences.

Mistake 6: Dropping the k term too early

The substitution pattern is:

$$T(n) = 2^k T(n / 2^k) + kn$$

The kn part is very important.

Since:

$$k = \log_2(n)$$

then:

$$kn = n \log_2(n)$$

So the final answer becomes:

$$O(n \log n)$$

Mistake 7: Thinking stack space is $O(n \log n)$

The time is:

$$O(n \log n)$$

But stack space is:

$$O(\log n)$$

Why?

The deepest active path is only:

```
n → n/2 → n/4 → ... → 1
```

That path has `log n` calls.

30. Step-by-Step Checklist for Phase 8 Problems

When you see a Phase 8-style recursive function, follow these steps.

Step 1: Find the base case

Look for the stopping condition.

Example:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Step 2: Count recursive calls

Example:

```
algorithm(n / 2);  
algorithm(n / 2);
```

There are two recursive calls.

So:

a = 2

and the recursive part has:

$2T(\dots)$

Step 3: Track how the input changes

The input changes from:

n

to:

$n / 2$

So the recursive part is:

$2T(n / 2)$

Step 4: Count extra work

Example:

```
for (int i = 1; i <= n; i++) {  
    cout << i << " ";  
}
```

The loop runs times.

So the extra work is:

+n

Step 5: Write the recurrence

$$\begin{aligned}T(n) &= 2T(n / 2) + n \\T(1) &= 1\end{aligned}$$

Step 6: Substitute repeatedly

$$\begin{aligned}T(n) &= 2T(n / 2) + n \\T(n) &= 2^2T(n / 4) + 2n \\T(n) &= 2^3T(n / 8) + 3n\end{aligned}$$

Step 7: Find the pattern

$$T(n) = 2^k T(n / 2^k) + kn$$

Step 8: Use the base case

$$n / 2^k = 1$$

Step 9: Solve for k

$$\begin{aligned}n &= 2^k \\k &= \log_2(n)\end{aligned}$$

Step 10: Substitute back

$$T(n) = 2^k T(1) + kn$$

Since:

$$\begin{aligned}2^k &= n \\ k &= \log_2(n) \\ T(1) &= 1\end{aligned}$$

we get:

$$T(n) = n + n\log_2(n)$$

Step 11: Write the final Big O

$$T(n) = O(n \log n)$$

Stack space:

$$O(\log n)$$

31. Mini Practice Example 1

Find the recurrence and time complexity.

```
void test(int n) {  
    if (n > 1) {  
        for (int i = 1; i <= n; i++) {  
            cout << "*";  
        }  
        test(n / 2);  
        test(n / 2);  
    }  
}
```

Solution

Base case:

$$T(1) = 1$$

There are two recursive calls:

```
test(n / 2);  
test(n / 2);
```

So the recursive part is:

```
2T(n / 2)
```

The loop runs times, so the current work is:

```
+n
```

Recurrence:

```
T(n) = 2T(n / 2) + n
```

Time complexity:

```
O(n log n)
```

Stack space:

```
O(log n)
```

32. Mini Practice Example 2

Find the recurrence and time complexity.

```
void divideAndPrint(int n) {  
    if (n > 1) {  
        for (int i = 0; i < n; i++) {  
            cout << i;  
        }  
        divideAndPrint(n / 2);  
        divideAndPrint(n / 2);  
    }  
}
```

Solution

The function stops when:

$$n = 1$$

So:

$$T(1) = 1$$

The function has two recursive calls.

Each recursive call receives:

$$n / 2$$

So the recursive part is:

$$2T(n / 2)$$

The loop runs times:

```
for (int i = 0; i < n; i++)
```

So the extra work is:

$$+n$$

Final recurrence:

$$\begin{aligned} T(n) &= 2T(n / 2) + n \\ T(1) &= 1 \end{aligned}$$

Final time:

$$O(n \log n)$$

Stack space:

$O(\log n)$

33. Mini Practice Example 3

Find the recurrence only.

```
void example(int n) {
    if (n > 1) {
        for (int i = 1; i <= n; i++) {
            cout << "Hello";
        }
        example(n / 2);
        example(n / 2);
    }
}
```

Solution

There are two recursive calls:

$2T(n / 2)$

There is linear work:

$+n$

Base case:

$T(1) = 1$

Recurrence:

$T(n) = 2T(n / 2) + n$
 $T(1) = 1$

Time complexity:

$O(n \log n)$

34. Simple Way to Remember Phase 8

For this code pattern:

```
function(n) {  
  if (n > 1) {  
    do n work;  
    function(n / 2);  
    function(n / 2);  
  }  
}
```

The recurrence is:

$T(n) = 2T(n / 2) + n$

The time complexity is:

$O(n \log n)$

The stack space is:

$O(\log n)$

Memory trick 1

Two half-size calls plus linear work gives $O(n \log n)$.

Memory trick 2

Each level costs n .
There are $\log n$ levels.
So total work is $n \log n$.

Memory trick 3

$$T(n) = 2T(n / 2) + 1 \rightarrow O(n)$$
$$T(n) = 2T(n / 2) + n \rightarrow O(n \log n)$$

35. Why Phase 8 Is Important

Phase 8 is important because it appears often in real algorithms.

The most famous example is **merge sort**.

Any algorithm with this pattern often gives $O(n \log n)$ time:

Split into two half-size problems.
Solve both halves recursively.
Do linear work to combine or process the result.

This pattern is common in divide-and-conquer algorithms.

36. Final Phase 8 Summary

In Phase 8, you learned about dividing recursive functions with two recursive calls and linear work.

The main code was:

```
void algorithm(int n) {  
    if (n > 1) {  
        for (int i = 1; i <= n; i++) {  
            cout << i << " ";  
        }  
        algorithm(n / 2);  
        algorithm(n / 2);  
    }  
}
```

The function:

1. Runs a loop from 1 to n.
2. Calls itself with $n / 2$.
3. Calls itself again with $n / 2$.

4. Stops when $n = 1$.

The recurrence is:

$$\begin{aligned}T(n) &= 2T(n / 2) + n \\T(1) &= 1\end{aligned}$$

Using substitution:

$$\begin{aligned}T(n) &= 2T(n / 2) + n \\T(n) &= 2^2T(n / 4) + 2n \\T(n) &= 2^3T(n / 8) + 3n\end{aligned}$$

Pattern:

$$T(n) = 2^kT(n / 2^k) + kn$$

Base case equation:

$$n / 2^k = 1$$

Solve:

$$\begin{aligned}n &= 2^k \\k &= \log_2(n)\end{aligned}$$

Substitute back:

$$\begin{aligned}T(n) &= 2^kT(1) + kn \\T(n) &= n + n\log_2(n)\end{aligned}$$

Final time complexity:

$$O(n \log n)$$

Stack space:

$$O(\log n)$$

The most important lesson is:

$T(n) = 2T(n / 2) + n$ means each level costs n , and there are $\log n$ levels.
So the answer is $O(n \log n)$.

37. What You Should Be Able to Do After Phase 8

After Phase 8, you should be able to:

1. Recognize a dividing recursive function with two recursive calls.
2. Recognize linear work inside a recursive function.
3. Understand that a loop from 1 to n gives $+n$ work.
4. Understand that two calls to $\text{algorithm}(n / 2)$ give $2T(n / 2)$.
5. Write the recurrence $T(n) = 2T(n / 2) + n$.
6. Identify the base case $T(1) = 1$.
7. Understand that the recursion tree has $\log n$ levels.
8. Understand that every level costs n total work.
9. Explain why the total work is $n \log n$.
10. Solve the recurrence using successive substitution.
11. Use the base case equation $n / 2^k = 1$.
12. Solve for $k = \log_2(n)$.
13. Substitute back to get $T(n) = n + n \log_2(n)$.
14. Conclude that the time complexity is $O(n \log n)$.
15. Understand why the stack space is $O(\log n)$.
16. Connect this recurrence to merge sort.
17. Avoid the mistake of saying the answer is only $O(n)$.
18. Avoid the mistake of saying the answer is $O(n^2)$.
19. Avoid the mistake of saying every dividing recurrence is $O(\log n)$.
20. Remember that $2T(n / 2) + n$ is one of the most important divide-and-conquer patterns.

38. Key Formulas From Phase 8

Main recurrence

$$\begin{aligned} T(n) &= 2T(n / 2) + n \\ T(1) &= 1 \end{aligned}$$

After substitution

$$T(n) = 2^k T(n / 2^k) + kn$$

Base case equation

$$n / 2^k = 1$$

Solve for k

$$\begin{aligned} n &= 2^k \\ k &= \log_2(n) \end{aligned}$$

Substitute back

$$T(n) = 2^k T(1) + kn$$

Since:

$$\begin{aligned} 2^k &= n \\ k &= \log_2(n) \\ T(1) &= 1 \end{aligned}$$

Then:

$$T(n) = n + n \log_2(n)$$

Final result

Time: $O(n \log n)$
Space: $O(\log n)$

Most important memory sentence

Two recursive calls on half input plus linear work gives $O(n \log n)$.

39. Final Memory Table

Recurrence	Meaning	Final time
$T(n) = T(n / 2) + 1$	One half-size call, constant work	$O(\log n)$
$T(n) = T(n / 2) + n$	One half-size call, linear work	$O(n)$
$T(n) = 2T(n / 2) + 1$	Two half-size calls, constant work	$O(n)$
$T(n) = 2T(n / 2) + n$	Two half-size calls, linear work	$O(n \log n)$

The Phase 8 recurrence is:

$$T(n) = 2T(n / 2) + n$$

Final answer:

Time complexity = $O(n \log n)$
Stack space = $O(\log n)$

End of Phase 8 Structured Note